ANALYST INTERN PROJECT · NBA TRACKING DATA

PREDICTING NBA SHOT MAKE PROBABILITY

Diamond Rogers

0.6265	0.6896	425,719	20
Log-loss on an unseen season	League-average baseline	Training shots, two seasons	Features in the model

SUMMARY

I built a gradient-boosted tree model that estimates the probability an NBA shot goes in, using twenty features derived from the provided tracking data. Evaluated against a season the model had never seen, it achieves a log-loss of 0.6265, compared to 0.6896 for a model that predicts the league-average make rate on every shot.

The most consequential decision in this project was not which algorithm to use. It was how to validate it, and I give that its own section below, because I think it matters more than anything else here.

THE DATA, AND A PROBLEM IN IT

The training set contains 425,719 non-fouled regular season shots across 29 columns; the testing set contains 213,977 shots with the same features minus the outcome. Across the training data, 45.8% of shots went in.

Loading the data produced a parsing warning, which I followed up rather than dismissed. `problems()` returned 74 failures, all in columns 12 and 16 (`contester4_id`

and `distcont4`), where R expected a logical value and instead found a player ID and a distance in feet.

The cause is that readr infers each column's type from the first 1,000 rows. Both of those columns are empty until row 4,023, so the sample R used contained nothing but NA. An all-NA column looks like a logical column, so that is what R assumed, and every real value appearing later in the file was discarded as a type mismatch. The fix was to set `guess_max = 1000000`, forcing R to inspect the full file before deciding. Both columns then typed correctly and the warning disappeared.

The practical impact here was small: `distcont4` is populated on well under 1% of shots, so little information was actually lost. But I would not have known that without checking, and a silent type coercion that discards data is exactly the kind of failure that does not announce itself.

WHAT SEPARATES A GOOD SHOT FROM A BAD ONE

Before modeling, I wanted to understand what the data could and could not distinguish.

Shot type carries an enormous spread. Dunks go in 88.5% of the time; heaves, the desperation launches at the buzzer, go in 9.5%. That is nearly eighty percentage points between the best and worst category.

But shot type alone is worth far less than that spread suggests. A model that predicts each shot type's historical make rate and nothing else scores a log-loss of 0.654, an improvement of only 0.036 over the base rate. The reason is visible in the shot counts: jumpers are 231,542 of the 425,719 shots in the data. More than half the dataset sits in a single category whose make rate, 37.8%, is close to the overall average. The dramatic numbers live at the edges; the volume lives in the middle. So the real question is what separates one jumper from another.

Distance helps, but less than expected. Within jump shots, make rate declines cleanly from 52.8% inside eight feet to 26.5% beyond thirty, about twelve points across the meat of the distribution. More importantly, two-thirds of all jumpers fall in a narrow band between 22 and 27 feet, which is the three-point line. Those two bins make 38.5% and

36.1% respectively. Distance can barely separate the single largest cluster of shots in modern basketball.

Defender proximity helps by a similar amount. A jumper with a defender inside two feet goes in 30.7% of the time; with nobody within eight feet, 40.8%. About ten points.

> A completely unguarded NBA jump shot goes in 40.8% of the time.

That number is worth sitting with. Shooting is dominated by variance, and no model built on this data is going to call individual shots well. The achievable range for log-loss here is in the low 0.60s, not the 0.40s, and it is worth being honest about that up front rather than treating it as a shortfall.

Combining features is where the value is. Crossing shot distance against defender distance produces a spread from 23.5% (beyond 27 feet, defender inside four feet) to 54.0% (inside sixteen feet, unguarded): thirty points, from two features worth ten and twelve individually.

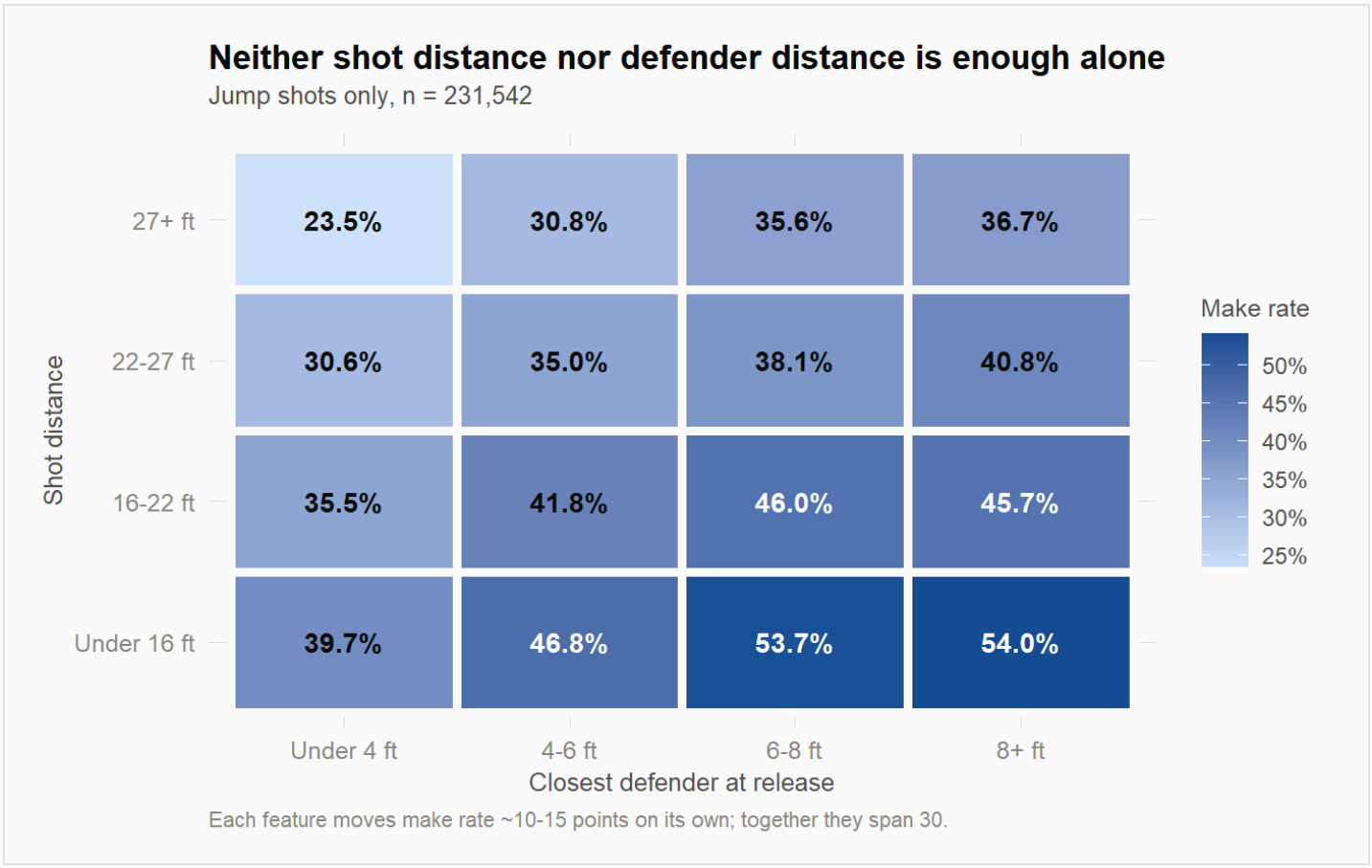


Figure 1. Make rate for jump shots by shot distance and closest-defender distance at release.

I checked whether that came from an interaction between the two features, and it does not. The defender effect is +14.3 points inside sixteen feet and +13.2 points beyond twenty-seven; the distance effect is −16.2 points when guarded and −17.3 points when open. Those are close to additive. Being closely defended costs roughly the same amount whether you are at the rim or at thirty feet.

I had expected to find an interaction and did not. The thirty-point spread comes from stacking independent signals rather than from any one of them being conditional on another. That is useful to know, because it means the path to a better model is adding more weak-but-independent features, not hunting for clever crossings.

THE MODEL

I used gradient-boosted trees (xgboost, binary logistic objective, log-loss evaluation), for two reasons specific to what I found above.

First, the relationship between distance and make probability is non-linear: steep near the rim, then nearly flat across the entire three-point band. A linear model would need me to specify that shape by hand. A tree model finds it. Second, the value in this data comes from combining roughly twenty weak signals. Trees do that without my having to pre-select which ones matter or how they should be weighted.

Features used (20 total)

Shot geometry	`distance`, `locationx`, `locationy`, `three`
Defensive pressure	`closestdefdist`, `distcont1`, `num_contesters`, `contested`
Shot context	`shotclock`, `dribblesbefore`, `shooterspeed`
Categorical indicators	One binary column per shot type (jumper, layup, dunk, floater, post, tip, heave) and per game state (transition, direct second chance)

I expanded the categorical variables into explicit 0/1 indicators rather than integer codes, so the model could not read an arbitrary numbering (dunk = 3, floater = 4) as an ordering that does not exist.

Deliberately excluded: `season_id`. The test set is a season that appears nowhere in training, so any pattern the model learned about the two training seasons would have no test-time counterpart. Including it would only invite the model to fit season-specific noise.

Parameters: `max_depth = 6`, `eta = 0.1`, `subsample = 0.8`, `colsample_bytree = 0.8`, with early stopping on validation log-loss after 30 rounds without improvement. Training stopped at iteration 101 and retained iteration 71.

VALIDATION: THE DECISION THAT MATTERED MOST

The training data contains two seasons, `2676a` (212,458 shots) and `fe055` (213,261). The testing data is a third season, `d7a2d`, which appears in neither. This is a temporal holdout, not a random one.

That changes what an honest validation looks like. Under a random 80/20 split, a held-out row might be the very next shot of the same possession as a training row: same game, same night, same shooter against the same defense. The model would be validating against shots that are effectively siblings of what it trained on, and the resulting score would flatter it.

So I split by season instead: fit on `2676a`, validate on `fe055`. That reproduces the actual task (fit on one season, predict a season never seen) and gives a number I can trust to mean something about the test set. Having established the right number of rounds this way, I refit the final model on both seasons at that fixed count, so the submitted predictions use all available data rather than half of it.

RESULTS

Measured on the held-out season:

MODEL	LOG-LOSS
Coin flip (0.500 on every shot)	0.6931

MODEL	LOG-LOSS
League base rate (0.458 on every shot)	0.6896
Shot type only	0.6540
Full model	**0.6265**

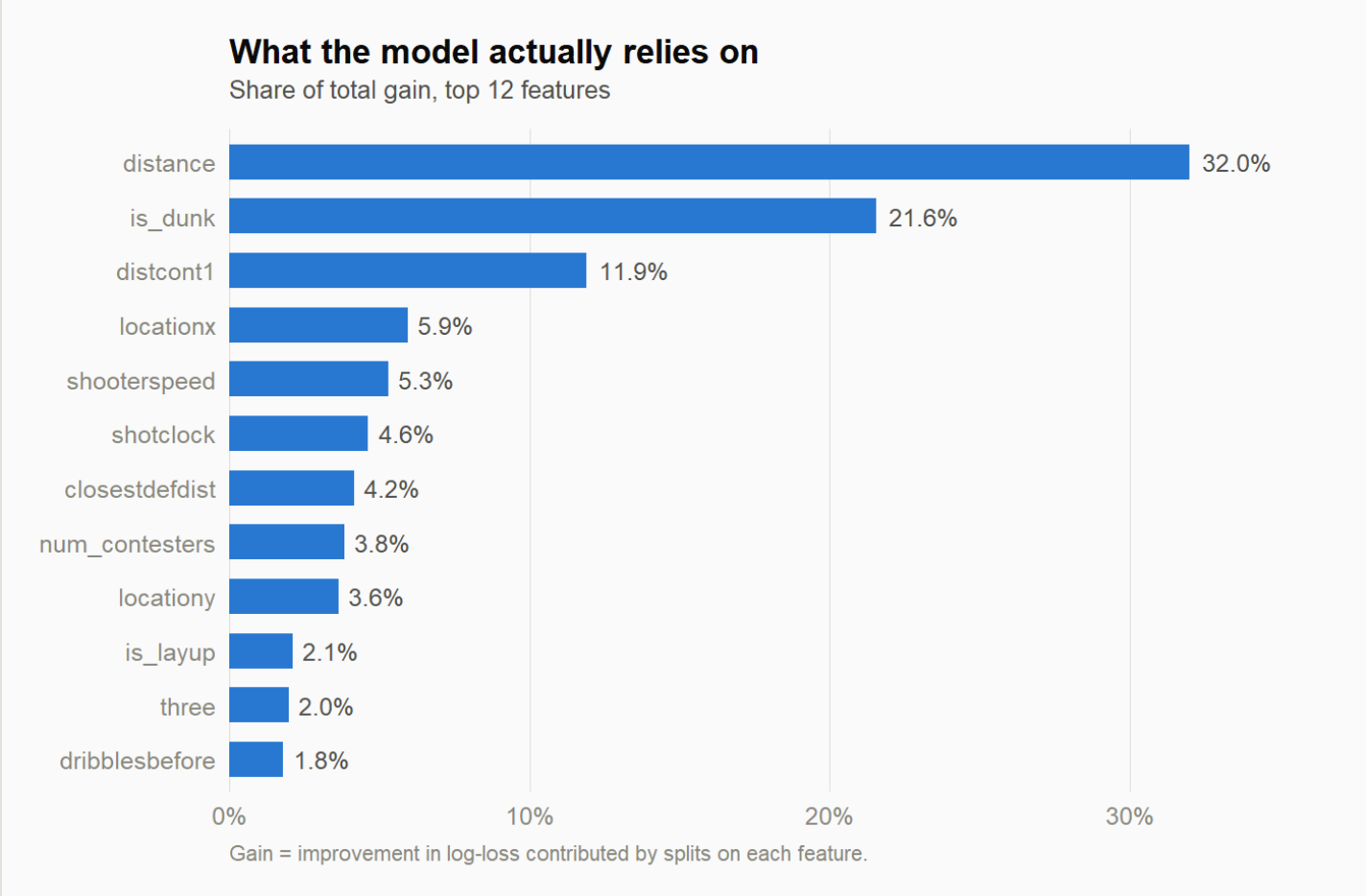


Figure 2. Relative importance of each feature, measured by share of total gain.

Two things in the importance ranking are worth calling out.

`distance` and `is_dunk` together account for 54% of total gain. `is_dunk` earns 21.6% of the gain while being used in under 1% of splits. It is rare but decisive, which is what you would expect from an 88.5% make rate sitting in a sea of 45% shots.

The contester's distance is roughly three times as informative as the closest defender's: `distcont1` at 11.9% gain against `closestdefdist` at 4.2%. The brief notes that the closest defender may or may not be labeled as a contester, and this result suggests that distinction is doing real work. A defender three feet away who is trailing

the play, turned around, or guarding someone else is close but is not bothering the shot. The contester is the player actually challenging it, and the model weights him accordingly.

A smaller observation in the same vein: the boolean `contested` flag contributes 0.1%, essentially nothing. Not because contesting does not matter, but because `distcont1` and `num_contesters` already express it with far more precision. A yes/no flag adds nothing once you know the actual distance.

The model is not overfitting. Training log-loss finished at 0.618 against 0.6265 on validation, a gap of under 0.01. An overfit model shows a widening gap as training loss falls and validation loss climbs. This one barely separates, which suggests it is closer to underfitting than overfitting and that added capacity or added features would likely help rather than hurt.

ARE THE PROBABILITIES HONEST?

Log-loss measures how wrong the predictions are. It does not directly tell you whether a stated probability means what it says. Since this project is scored against another model's probabilities rather than against realized outcomes, calibration seemed worth checking explicitly. I grouped the validation-season predictions into twenty equal bins and compared the mean predicted probability in each against the observed make rate.

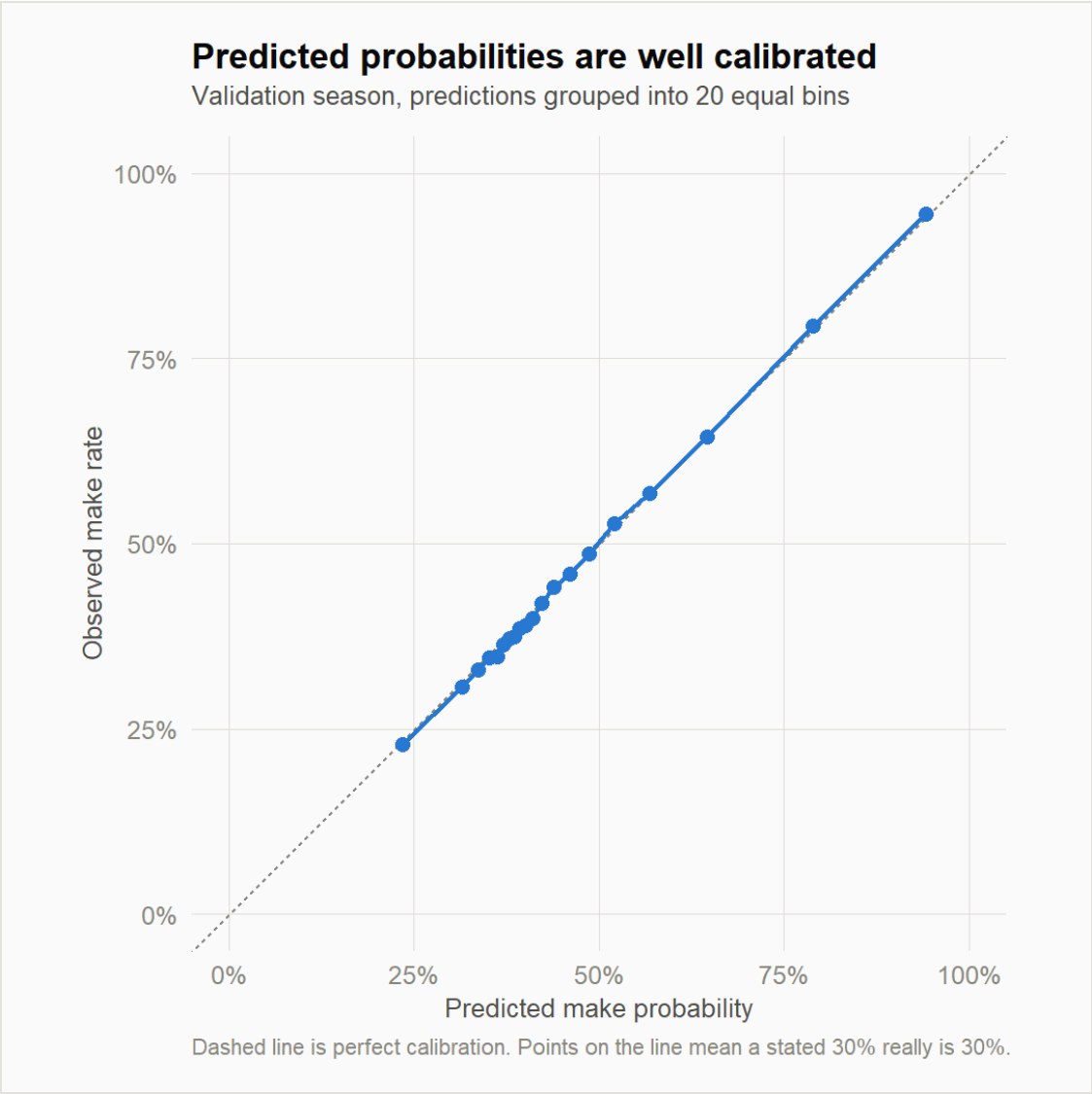


Figure 3. Predicted probability against observed make rate, validation season, twenty equal-sized bins.

The points track the diagonal closely across the full range. When this model says 30%, roughly 30% of those shots go in; when it says 70%, roughly 70% do. The predictions are usable as probabilities and not merely as a ranking.

LIMITATIONS, AND WHAT I WOULD DO NEXT

`closestdefapproach` is unused, and it is the most promising thing left.

The column holds the closest defender's distance at 1.00, 0.75, 0.50, and 0.25 seconds before release, which combined with `closestdefdist` describes the defender's entire closing trajectory. My model sees only a snapshot at release, so a defender six feet away and flat-footed is indistinguishable from one six feet away and closing hard. Given

that contester distance turned out to be the third-most-valuable feature, I would expect closing speed to carry real signal. I would derive it and test it first.

Shooter identity is unused.

`shooter_id` is available and players differ substantially in shooting ability. The complication is that a raw player indicator across hundreds of shooters would overfit badly on small samples, so this needs a regularized treatment: an empirical-Bayes shrunk shooting rate computed on training data only, to avoid leaking outcome information into the features. That is a careful piece of work rather than a quick addition, which is why I left it out of a first model rather than doing it badly. Defensive team identity is unused for similar reasons.

Model capacity is likely under-used.

The small train/validation gap suggests room for deeper trees, a lower learning rate over more rounds, or both. I would tune these against the season-split validation rather than a random one, for the reasons above.

What more data would buy.

Additional seasons would help most with the player-level features, where the limiting factor is sample size per shooter rather than total shot count. Shot-arc or release-angle data would likely add more than anything derivable from what is here, since two shots identical in every recorded dimension can differ entirely in how well they were actually struck, and that unmeasured difference is a large part of the residual noise this model cannot reach.

The honest ceiling.

A wide-open NBA jump shot goes in 40.8% of the time. Most of what determines whether a given shot falls is not in this dataset and probably is not in any dataset. A model here should be judged on how well it sorts shots by quality and on whether its probabilities are trustworthy, not on how close to certainty it can get.